Title:
Neural Network Builder API Refactor

Abstract:

sbi's neural network factory functions have grown organically, leading to late validation, mixed concerns, and opaque configuration that make the codebase harder to use and maintain.

This project refactors the builder infrastructure by introducing typed builder classes (DensityEstimatorBuilder, ClassifierBuilder, VectorFieldEstimatorBuilder) that validate configuration at construction time instead of at build time, enabling earlier error detection and safer usage.

A clear separation between user intent (ZScoreConfig) and resolved artifacts (ZScoreContext) decouples z-score computation from model construction. Trainers are responsible for resolving z-scoring from data, while builders consume only prepared artifacts via a BuildContext, enforcing a clean abstraction boundary.

The refactored API provides three levels of control:

- a string-based high-level API for quick experimentation,
- builder objects with inspectable and serializable configuration for production use,
- and direct nn.Module passing for full custom control.

Legacy factory functions (posterior_nn, likelihood_nn, classifier_nn, posterior_flow_nn, posterior_score_nn) are preserved with deprecation warnings, returning a LegacyBuilderBundle that ensures backward compatibility without silent data loss.

Trainer dispatch is updated across NPE, NLE, NRE, NPSE, and FMPE to support all API levels through a clean isinstance based mechanism. The design builds on existing configuration classes (_EstimatorConfigBase, ConditionalFlowConfig, FlowEstimatorConfig, ScoreEstimatorConfig) rather than replacing them.

Comprehensive tests will cover fail-fast validation, configuration serialization round-trips, and backward compatibility. A migration guide and updated documentation will support users in transitioning to the new API.

Technical Details:

sbi (simulation-based inference) is a PyTorch-based library for posterior estimation using neural networks. Its factory functions (‘posterior_nn’, ‘likelihood_nn’, ‘classifier_nn’, ‘posterior_flow_nn’, ‘posterior_score_nn’,’marginal_nn’) currently return opaque callables that are passed to inference trainers (NPE, NLE, NRE, FMPE,NPSE). This pattern has three core problems:

The existing sbi neural network factory functions have several key issues that affect usability, maintainability, and reproducibility:

1. **Late validation (errors surface at the wrong time.)**
   A user who calls, for example, `posterior_nn(model="nsf", num_bins="ten")` receives no error at configuration time. The error only surfaces when training begins and the actual build function is invoked with real data. For a training run that takes hours to reach the build step, this is a significant usability failure. Similar issues occur when passing `num_components` (MDN-only) to an NSF model, or spline-specific fields to a MAF; these are silently ignored or fail late.

2. **Mixed concerns (z-scoring is fused into model construction.)**
   In the current design, z-score statistics (means, standard deviations, input transforms) are computed from training data at the same time as the neural network weights are initialized. This couples data normalization and model architecture in a single opaque step. There is no way to inspect or test the normalization independently, or to reuse the same normalization across multiple model builds. The proposed `ZScoreConfig → ZScoreContext` separation captures user intent separately from resolved artifacts, untangling these concerns.

3. **Opaque configuration (builders cannot be inspected or serialized.)**
   Factory functions return closures whose configuration is stored in a closed-over local scope. There is no `builder.get_config()`, no `DensityEstimatorBuilder.from_config(config)`. This limitation complicates reproducibility and experimentation.

The Solution

This project refactors the neural network builder infrastructure around three core design principles:

1. Fail-fast validation

- Every call, e.g., `DensityEstimatorBuilder("nsf", num_bins="ten")`, validates immediately at construction time.
- Cross-model field validation (MDN-only fields, Zuko-only fields) is performed in `__init__`.
- Unknown `**kwargs` are rejected with descriptive errors.
- No configuration error can survive past object construction, preventing surprises during long training runs.

2. Separation of concerns

- Z-scoring is extracted from model construction into a dedicated resolver layer.
- The data flow is as follows:
  `ZScoreConfig` (user intent, immutable) → `resolve_z_transform()` (computed from data) → `ZScoreContext` (resolved artifacts, immutable) → `BuildContext` (full context passed to `build()`).
- This clean separation allows independent testing, reproducibility, and reuse of normalization across multiple builds.

3. Inspectable and serializable configuration

- `DensityEstimatorBuilder` wraps `ConditionalFlowConfig` (a typed dataclass).
- `get_config()` returns a plain dictionary.
- `from_config(config)` reconstructs the builder exactly.
- `__repr__` expose the full configuration.
- Builders can now be inspected, configured , serialized greatly improving transparency and reproducibility.

Layered API

The refactor introduces a three-tier API that preserves full backward compatibility while providing a clear upgrade path:

- **High-level (string):** `NPE(prior, density_estimator="maf")` — unchanged from current API, issues a `DeprecationWarning` internally when routed through `LegacyBuilderBundle`.
- **Mid-level (builder):** `NPE(prior, density_estimator=DensityEstimatorBuilder("nsf", hidden_features=64), z_score=ZScoreConfig(...))` — new recommended API with explicit, inspectable configuration.
- **Low-level (module):** `NPE(prior, density_estimator=MyCustomFlow())` — escape hatch for users with fully custom architectures who want to bypass the builder system entirely.

The trainer's `__init__` dispatches across these tiers via `isinstance` checks, and `check_estimator_arg` is updated to accept all valid types at each tier.

I have had prior conversations with mentor Jan Boelts (janfb) about this design, and have made contributions to the sbi codebase including merged PRs. A design prototype implementing the proposed dataclasses and resolver functions is available at [link to sbi-nn-api-refactor rep].

Relevant pieces of code that I want to change:

```
def classifier_nn(model: str, **kwargs):
    # Map user-facing parameter names to internal names
    config = ClassifierConfig.from_kwargs(**kwargs)
    builder_kwargs = config.to_dict()
    def build_fn(batch_theta, batch_x):
        if model == "linear":
            return build_linear_classifier(
                batch_x=batch_theta, batch_y=batch_x,
**builder_kwargs)
        if model == "mlp":
            return build_mlp_classifier(
                batch_x=batch_theta, batch_y=batch_x,
**builder_kwargs)
    return build_fn
```

Current behavior / problem:

- The function returns a callable that builds a neural network, instead of a structured builder object.

- Validation of parameters happens only at build time, not at construction, which can lead to late errors during training.
- Configuration is not inspectable or serializable, preventing reproducibility and complicating debugging.

Proposed change / improvement:

- Introduce a `DensityEstimatorBuilder` class that encapsulates configuration and validation.
- Validation happens at builder construction, not at build time.
- Builders store config in an inspectable and serializable format, ensuring reproducibility and clean trainer integration.

```
def build_maf(
    batch_x: Tensor,
    batch_y: Tensor,
    z_score_x: Literal[
        "none", "independent", "structured", "transform_to_unconstrained"
    ] = "independent",
    z_score_y: Literal[
        "none", "independent", "structured", "transform_to_unconstrained"
    ] = "independent",
    **kwargs,
) -> NFlowsFlow:
    z_score_x_bool, structured_x = z_score_parser(z_score_x)
    if z_score_x_bool:
        transform_list = [
            standardizing_transform(batch_x, structured_x)
        ] + transform_list
    embedding_net = _prepare_y_embedding(z_score_y, batch_y, embedding_net)
    # Combine transforms
    transform = transforms.CompositeTransform(transform_list)
    distribution = get_base_dist(x_numel, **kwargs)
    neural_net = flows.Flow(transform, distribution, embedding_net)
    flow = NFlowsFlow(
        neural_net, input_shape=batch_x[0].shape,
condition_shape=batch_y[0].shape
    )
    return flow
```

Description / Problem:

1. Z-scoring logic is mixed with model construction → not modular or testable.
2. Validation happens late , only when the function is called.

Refactor justification:

- Introduce `BuildContext` to hold tensors, shapes, device, dtype.
- Separate z-scoring via `ZScoreConfig` / `ZScoreContext`.

Libraries Used in This Project

- **torch** – Provides tensors, `nn.Module`, and handles device/dtype management across all builders and trainers.
- **nflows** – Used to build normalizing flows (MAF, NSF, MAF-RQS) and to resolve z-score transforms for nflows-based models.
- **zuko** – Builds Zuko-based flows (e.g., `zuko_maf`, `zuko_nsf`) and resolves z-score transforms for Zuko models.
- **torch.distributions** – Type for `ZScoreConfig.x_dist`; used when computing unconstrained transforms for Zuko models.
- **dataclasses** – Defines all frozen config/context dataclasses, including `ZScoreConfig`, `BuildContext`, and `LegacyBuilderBundle`.
- **abc** – Provides the `EstimatorBuilder` abstract base class with `@abstractmethod`.
- **warnings** – Emits `DeprecationWarning` when legacy factory functions (e.g., `posterior_nn`, `likelihood_nn`) are called.
- **typing** – Used for type annotations across the project (`Literal`, `Optional`, `Union`, `Protocol`).
- **pytest** – Framework for all unit and integration tests.

Schedule of deliverables

Implemented So Far

This prototype already includes working implementations of the core normalization infrastructure:

- ✅ **ZScoreConfig** — user-declared normalization intent, including x, theta, and x_dist (required for transform_to_contrained option in Zuko).
- ✅ **Normalization dataclasses** — StatsNormalization, TransformNormalization, InputTransformNormalization.
- ✅ **ZScoreContext hierarchy** — base context, ClassifierZScoreContext, MixedZScoreContext.

- ✅ **Resolver functions** — implemented for all estimator families.
- ✅ **BuildContext abstraction** — bundles shape info, z-score stats, device, dtype, and raw tensors (batch_x, batch_theta).

These components form the **configuration → resolution pipeline**, which is the foundation of the proposed neural network API refactor.

Key Implementation Notes

1. **x_dist in ZScoreConfig**
   - Only required for transform_to_contrained in Zuko.
   - Included here because it is needed for z-score calculation, not for model building.
2. **Inclusion of batch_x and batch_theta in BuildContext**
   - For standard backends (nflows, Zuko, MDN), x_numel and y_numel could be precomputed in the trainer , raw tensors are not strictly necessary.
   - **MixedDensityEstimatorBuilder** requires raw tensors inside build() for additional purposes:
     - batch_x is used by _is_discrete() and _separate_input() to split continuous and discrete columns.
     - batch_theta is used to compute combined_condition = cat([disc_x, embedded_y]), which determines the combined embedding network input size.
   - Including raw tensors keeps BuildContext clean and **build() signature uniform** across all builders. No special casing or extra fields for mixed-specific logic are required.

Week	Deliverables
1. Week 1	Apply requested Changes in implemented code and unit tests
2. Week 2	Implement `EstimatorBuilder` ABC with abstract methods and updated typechecks
3. Week 3	Build `DensityEstimatorBuilder` integrating mixed density estimation (MNLE/MNPE)
4. Week 4	Continue/complete `DensityEstimatorBuilder` and add resolver function and it's tests

5. Week 5	Implement `ClassifierBuilder` class
6. Week 6	Implement `VectorFieldEstimatorBuilder` class
7. Week 7	Deprecate legacy factory functions; implement `LegacyBuilderBundle`; update typechecks
8. Week 8	Integrate new builders with trainers; `_initialize_neural_network()` builder.build() logic
9. Week 9	Integration tests and backward compatibility validation
10. Week 10	Update documentation and tutorials; create migration guide
11. Week 11	Apply final feedback
12. Week 12	Push all code

Phase 1

Week 1: Apply Mentor-Requested Changes

- Apply mentor-requested changes on existing code and tests for the following modules:
  - **Normalization**
  - **ZScoreConfig**
  - **ZScoreContext**
  - **BuildContext**
- Ensure that all unit tests reflect the updated logic.
- Maintain code consistency and readability while applying changes.

Week 2: Build EstimatorBuilder (Abstract Base Dataclass for All Estimator Builders)

Abstract Methods

The following methods need to be defined as abstract methods in the `EstimatorBuilder` base class:

- **`build`**
  - Builds the estimator.
- **`resolve_z_transform`**
  - Handles z-score transformations as required by the builder.
- **`get_config`**
  - Returns a serializable dictionary of the builder's configuration.
- **`validate_with_zscore_config`**
  - Checks if the z-score configuration is incompatible with the builder.
  - Called during initialization of the trainer for validation before training.
- **`__call__`**
  - Allows the builder to be called like a function.
- **`from_config`**
  - Reconstructs the builder from a configuration dictionary returned by `get_config`.

Static Methods

- **`resolve_embedding_net`**
  - Required by every builder class.
  - Defined as a `@staticmethod`.

Other Updates

- Update type checks to include `EstimatorBuilder`.

Unit Tests

- Test the `EstimatorBuilder` abstract base class (ABC) contract.
- Test typecheck updates for the builder classes.

Week 3 & 4: Build DensityEstimatorBuilder Class

- **Objective:**
  Build the **`DensityEstimatorBuilder`** class for the **sbi neural network API refactor**, supporting mixed density estimation (MNLE/MNPE).
- **Mixed Density Handling:**
  - `MixedDensityEstimator` logic is handled **within** **`DensityEstimatorBuilder`** via the existing `mixed_nets.py` build functions.
  - No separate builder class is created for mixed density estimators.
- **Builder ↔ Config Hierarchy Mapping:**
  - **DensityEstimatorBuilder** wraps `ConditionalFlowConfig` (`self._config`)
  - **ClassifierBuilder** (week 5) wraps `ClassifierConfig`
  - **VectorFieldEstimatorBuilder** (week 6) wraps `ScoreEstimatorConfig` / `FlowEstimatorConfig`
  - Each builder delegates `**kwargs` to its config's `from_kwargs()` classmethod, inheriting validation and `to_dict()` serialization from `_EstimatorConfigBase`.
- **Class Design:**
  - Use a **plain Python class** (not `@dataclass`) because:
    - Accepts `**kwargs` and delegates to `ConditionalFlowConfig.from_kwargs()`.
    - `@dataclass` would require `init=False`, making `repr` and `eq` only see declared fields and hiding `_config` contents—misleading for inspection.
    - A plain class with handwritten `repr` and `eq` is strictly better for clarity and debugging.
- **Relevant Files:**
  - `sbi/neural_nets/net_builders/flow.py` — existing `nflows` / `zuko` logic
  - `sbi/neural_nets/net_builders/mdn.py` — existing MDN logic
  - `sbi/neural_nets/net_builders/mixed_nets.py` — existing MNLE / MNPE logic
- **Methods to Implement:**
  - `__init__`
  - All abstract methods from `EstimatorBuilder`

    - Static methods:
        - `resolve_nflows_input_transform`
        - `resolve_zuko_conditional_transforms`
        - `resolve_unconditional_zuko_transforms`
        - `resolve_mdn_transform_input`
        - `z_score_resolver_mixed_density_estimator`
- **Unit Tests:**
    - Builder construction
    - Config round-trip (`get_config` / `from_config`)
    - Cross-model field validation
    - Mixed model dispatch

Phase 2:

Week 5: Build ClassifierBuilder Class

- **Objective:**
  Build the **`ClassifierBuilder`** class as part of the sbi neural network API refactor.
- **Methods to Implement:**
    - `__init__`
    - All abstract methods from `EstimatorBuilder`
    - Static methods:
        - `resolve_zscore_classifier`
- **Unit Tests:**
    - Builder construction
    - Config wrapping
    - Resolver dispatch

Week 6: Build VectorFieldEstimatorBuilder

- **Objective:**
  Build the **`VectorFieldEstimatorBuilder`** class for the sbi neural network API refactor.
- **Methods to Implement:**
  - `__init__`
  - All abstract methods from `EstimatorBuilder`
  - Static methods:
    - `resolve_x_normalization_stats`
- **Unit Tests:**
  - Builder construction
  - Config wrapping
  - Resolver dispatch

Week 7: Legacy Factory Deprecation

- **Objective:**
  Deprecate legacy factory functions while maintaining backward compatibility and supporting low-level use cases.
- **Edits / Affected Files:**
  - `sbi/neural_nets/factory.py`
    - Functions: `posterior_nn`, `likelihood_nn`, `classifier_nn`, `posterior_flow_nn`, `posterior_score_nn`, `marginal_nn`
  - `sbi/utils/typechecks.py`
    - `check_estimator_arg` gains `Builder` and `LegacyBuilderBundle` `isinstance` checks
- **Changes to Factory Functions:**
  - Every factory function now constructs and returns a **`LegacyBuilderBundle`** (frozen dataclass wrapping builder + z_score_config).
  - Emits a **`DeprecationWarning`** when called.
- **Changes to Typechecks:**
  - `check_estimator_arg` now accepts `isinstance(estimator, LegacyBuilderBundle)` — the trainer unpacks the bundle into builder and z_score_config.
  - Also updated to accept `nn.Module` instances (previously rejected).

        - Supports low-level usage where users pass a pre-built `nn.Module` directly to the trainer, bypassing the builder.
- **Design Decisions:**
    - Returning builder only is insufficient — `z_score_theta` and `z_score_x` passed by users would be silently dropped, breaking backward compatibility.
    - Embedding z-score directly on builder is rejected — would require a hidden attribute (`_legacy_z_score_config`) that is not part of the public interface, making `get_config()` incomplete and `from_config()` reconstruction lossy.
    - **`LegacyBuilderBundle`** chosen to preserve full configuration and backward compatibility.

Week 8: Trainer Integration

- **Objective:**
  Integrate the refactored builders and z-score handling into the trainer classes across all inference types.
- **Edits / Affected Files:**
    - `sbi/inference/trainers/npe/npe_base.py`
    - `sbi/inference/trainers/nle/nle_base.py`
    - `sbi/inference/trainers/nre/nre_base.py`
    - `sbi/inference/trainers/vfpe/base_vf_inference.py`
    - `sbi/inference/trainers/base.py`
- **`__init__` Updates — `isinstance` Dispatch:**

```
 if isinstance(density_estimator, str):
    # High-level -- user passes only model name string
    # z_score never passed in high-level API -- always use default
ZScoreConfig()
    self._builder = DensityEstimatorBuilder(model=density_estimator)
    self._z_score_config = ZScoreConfig()  # defaults:
theta="independent", x="independent"

elif isinstance(density_estimator, LegacyBuilderBundle):
    # Legacy factory -- posterior_nn() returns LegacyBuilderBundle
```

```
    # z_score_config extracted from bundle -- preserves user's
original intent
    self._builder = density_estimator.builder
    self._z_score_config = density_estimator.z_score_config

elif isinstance(density_estimator, EstimatorBuilder):
    # Mid-level -- new API
    # z_score passed explicitly by user via trainer kwarg
    self._builder = density_estimator
    self._z_score_config = z_score if z_score is not None else
ZScoreConfig()

elif isinstance(density_estimator, nn.Module):
    # Low-level -- direct module, no building needed
    self._neural_net = density_estimator
    self._builder = None
    self._z_score_config = None
_initialize_neural_network() Updates -- Call Builder:

 z_score_context = self._builder.resolve_z_transform(batch_x,
batch_theta, self._z_score_config)

build_context = BuildContext(
    x_shape=batch_x.shape[1:],
    theta_shape=batch_theta.shape[1:],
    z_score_context=z_score_context,
    device=batch_x.device,
    dtype=batch_x.dtype,
    batch_x=batch_x,
    batch_y=batch_theta,
)

self._neural_net = self._builder.build(build_context)
```

- **Key Notes:**
  - High-level API uses only model name string; z-score defaults applied.
  - Legacy API unpacks `LegacyBuilderBundle` to preserve user-provided z-score config.

    - Mid-level API allows explicit builder + optional z-score config.
    - Low-level API accepts a raw `nn.Module` without using builder or z-score.

Week 9: Integration Tests and Backward Compatibility Validation

- **Objective:**
  Validate end-to-end integration of builders, trainers, and legacy factories, ensuring backward compatibility.
- **Edits / Affected Files:**
    - `tests/factory_config_test.py`
        - Extend existing tests to include **`LegacyBuilderBundle`** scenarios.
    - `tests/posterior_nn_test.py`
        - Backward compatibility: verify **factory** → **bundle** → **trainer** round-trip works correctly.
    - `tests/builder_integration_test.py`
        - End-to-end tests: **builder** → **trainer** → **train** → **posterior** workflow.
- **Key Notes:**
    - Unit tests for individual components are written alongside each week's deliverables.
    - Week 9 focuses on **cross-component integration tests** and **backward compatibility validation**, requiring multiple previous weeks' deliverables to be in place.

Week 10: Documentation Updates

- **Objective:**
  Update documentation to reflect the refactored builders, legacy factory deprecation, and migration path.
- **Edits / Updates:**
    - `docs/advanced_tutorials/03_density_estimators.ipynb` — update examples.
    - `docs/how_to_guide/03_density_estimators.ipynb` — update examples.
- **New Documentation:**

- `docs/how_to_guide/24_migration_guide.md` — create a **new migration guide** for transitioning from legacy factories to the new builder-based API.

Week 11: Final Feedback and Implementation

- Objective:
  Incorporate final feedback from mentors, reviewers, or stakeholders, and implement all requested changes across code, tests, and documentation.
- Key Actions:
  - Review feedback for code, unit tests, integration tests, and documentation.
  - Apply necessary updates and corrections.
  - Ensure consistency, correctness, and completeness across all modules.
  - Validate that all builders, trainers, legacy factory handling, and documentation reflect the requested changes.

Week 12: Push Code

- **Objective:**
  Finalize the project and push all code, tests, and documentation to the repository.
- **Key Actions:**
  - Ensure all unit tests and integration tests pass.
  - Confirm documentation is up to date.
  - Perform final code cleanup and formatting.
  - Push all changes to the main branch / repository.

Development Experience:

Contributions to sbi:

https://github.com/sbi-dev/sbi/pull/1794 : Refactored embedding net tests to use shared helpers, verify parameter updates, fix minor bugs, improve performance, and remove duplication — PR merged, issue #1506 closed.

https://github.com/sbi-dev/sbi/pull/1715 : Added optional GPU/MPS support to L-C2ST by implementing a PyTorch MLP via skorch, preserving sklearn-like defaults, enabling device overrides, and extending tests for CPU/GPU placement and user parameter overrides — PR merged, issue #1160 addressed.

https://github.com/sbi-dev/sbi/pull/1697 : Pinned PyMC to <5.20.1 and disabled its progress bar to fix TypeErrors in MCMC sampling, updated imports, and extended tests — PR merged, issue #1397 resolved.

https://github.com/Dev-Sudarshan/sbi-nn-refactor : A design prototype implementing the proposed dataclasses and resolver functions .

https://github.com/Dev-Sudarshan/Neural-Network-API-toolkit : modular PyTorch toolkit for building neural networks with a **layered API design**:

- **High-level:** `build_network()` for one-line model creation in common workflows.
- **Mid-level:** `MLPConfig` + `NetworkBuilder` for validated configurations and reusable builder patterns.
- **Low-level:** Accept custom `torch.nn.Module` for fully flexible architectures.

Other Experiences : Member of the Computer Science Club at Tribhuvan University , participated in workshops, coding events, and collaborative projects.

Why this project ?

I want to involve myself in activities that are complex and push my learning every day. The opportunity to code on a project used by many people excites me and motivates me to do my best. I specifically chose this project because it allows me to work on API design and neural networks and experience what it feels like to contribute to something meaningful and intricate. I am eager to collaborate, seek guidance, and contribute to a community that values knowledge. In the long term, I want to become an active open source contributor, and working with like minded people on this project will help me grow far more than working solo.

Personal details:

Name: Sudarshan Bhattarai

Email: sudarshanbhattarai04@gmail.com

Phone / WhatsApp: +977 9741695496

LinkedIn: linkedin.com/in/sudarshanbhattarai

X (Twitter): x.com/Sudarshan554

Education: Bachelor's student in Computer Science, Nepal